附录 第一问源程序

运行环境为 Python 3.10 和 R。Python 依赖 numpy、pandas、scipy、scikit-learn；R 依赖 ggplot2、ragg、dplyr、tidyr、readr、patchwork、svglite。全部模型程序、固定划分和绘图数据随支撑材料提供，具体命令见程序目录 README。以下按文件名列出源程序；Word 中的自动换行不改变支撑材料文件。

baseline_models.py

```
"""Small, testable building blocks for the Q1 CPU baselines."""

from __future__ import annotations

from collections.abc import Callable, Sequence
from typing import Any

import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import MissingIndicator, SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.preprocessing import QuantileTransformer, StandardScaler
from sklearn.model_selection import KFold


def fit_predict_global_mean(y_train: np.ndarray, n_validation: int) -> np.ndarray:
    """Predict each target with its training-fold mean."""
    mean = np.asarray(y_train, dtype=float).mean(axis=0, keepdims=True)
    return np.repeat(mean, repeats=n_validation, axis=0)


def fit_predict_shrunk_style(
    y_train: np.ndarray,
    train_style: pd.Series,
    validation_style: pd.Series,
    shrinkage: float,
) -> np.ndarray:
    """Predict style means shrunk toward the training-fold global mean."""
    y_train = np.asarray(y_train, dtype=float)
    train_style = pd.Series(train_style).reset_index(drop=True)
    validation_style = pd.Series(validation_style).reset_index(drop=True)
    if len(y_train) != len(train_style):
        raise ValueError("y_train and train_style have different lengths")
    if shrinkage < 0:
        raise ValueError("shrinkage must be non-negative")

    global_mean = y_train.mean(axis=0)
    predictions = []
```

```
    for style in validation_style:
        mask = train_style.eq(style).to_numpy()
        if not mask.any():
            predictions.append(global_mean)
            continue
        n_style = int(mask.sum())
        style_mean = y_train[mask].mean(axis=0)
        prediction = (n_style * style_mean + shrinkage * global_mean) / (
            n_style + shrinkage
        )
        predictions.append(prediction)
    return np.asarray(predictions)


def fit_predict_ridge(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    alpha: float,
) -> np.ndarray:
    """Fit a fold-local impute/scale/Ridge pipeline and predict all targets."""
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=alpha)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def fit_predict_pls(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_components: int,
) -> np.ndarray:
    """Fit a fold-local impute/scale/PLS2 pipeline and predict all targets."""
    max_components = min(len(x_train) - 1, np.asarray(x_train).shape[1])
    if not 1 <= n_components <= max_components:
        raise ValueError(
            f"n_components={n_components} outside valid range 1..{max_components}"
        )
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            ("model", PLSRegression(n_components=n_components, scale=True, max_iter=1000)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def _quantile_preprocessor(n_train: int, random_state: int) -> FeatureUnion:
    return FeatureUnion(
        [
            (
                "continuous",
                Pipeline(
                    [
```

```
                        ("imputer", SimpleImputer(strategy="median")),
                        (
                            "quantile",
                            QuantileTransformer(
                                n_quantiles=min(100, n_train),
                                output_distribution="normal",
                                subsample=None,
                                random_state=random_state,
                            ),
                        ),
                    ]
                ),
            ),
            (
                "missing_indicator",
                MissingIndicator(features="missing-only", error_on_new=False),
            ),
        ]
    )


def fit_predict_ridge_quantile(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    alpha: float,
    random_state: int,
) -> np.ndarray:
    """Ridge after a training-fold rank-Gaussian transform."""
    pipeline = Pipeline(
        [
            ("preprocessor", _quantile_preprocessor(len(x_train), random_state)),
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=alpha)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def fit_predict_pls_quantile(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_components: int,
    random_state: int,
) -> np.ndarray:
    """PLS2 after a training-fold rank-Gaussian transform."""
    max_components = min(len(x_train) - 1, np.asarray(x_train).shape[1])
    if not 1 <= n_components <= max_components:
        raise ValueError(
            f"n_components={n_components} outside valid range 1..{max_components}"
        )
    pipeline = Pipeline(
        [
            ("preprocessor", _quantile_preprocessor(len(x_train), random_state)),
            ("model", PLSRegression(n_components=n_components, scale=True, max_iter=1000)),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))
```

```
def fit_predict_extra_trees(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    n_estimators: int,
    max_features: float,
    min_samples_leaf: int,
    max_depth: int | None,
    random_state: int,
    n_jobs: int = -1,
) -> np.ndarray:
    """Fit a fold-local median-imputed multi-output ExtraTrees model."""
    pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            (
                "model",
                ExtraTreesRegressor(
                    n_estimators=n_estimators,
                    max_features=max_features,
                    min_samples_leaf=min_samples_leaf,
                    max_depth=max_depth,
                    random_state=random_state,
                    n_jobs=n_jobs,
                ),
            ),
        ]
    )
    pipeline.fit(x_train, y_train)
    return np.asarray(pipeline.predict(x_validation))


def empirical_cdf_grade(
    reference_targets: np.ndarray, predicted_targets: np.ndarray
) -> np.ndarray:
    """Map predictions to [0, 6] using training-only empirical CDFs."""
    reference_targets = np.asarray(reference_targets, dtype=float)
    predicted_targets = np.asarray(predicted_targets, dtype=float)
    if reference_targets.ndim != 2 or predicted_targets.ndim != 2:
        raise ValueError("reference_targets and predicted_targets must be 2-D")
    if reference_targets.shape[1] != predicted_targets.shape[1]:
        raise ValueError("reference and prediction target counts disagree")
    grades = np.empty_like(predicted_targets, dtype=float)
    for target_position in range(reference_targets.shape[1]):
        ordered = np.sort(reference_targets[:, target_position])
        ranks = np.searchsorted(
            ordered, predicted_targets[:, target_position], side="right"
        )
        grades[:, target_position] = 6.0 * ranks / len(ordered)
    return grades


def select_parameter_by_mae(
    targets: np.ndarray,
    candidates: Sequence[Any],
    predict_fold: Callable[[np.ndarray, np.ndarray, Any], np.ndarray],
    n_splits: int,
    random_state: int,
) -> tuple[Any, dict[Any, float]]:
```

```
    """Select one candidate using mean inner-fold multi-output MAE."""
    if not candidates:
        raise ValueError("candidates must not be empty")
    targets = np.asarray(targets, dtype=float)
    splitter = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    scores: dict[Any, float] = {}
    for candidate in candidates:
        fold_scores = []
        for train_index, validation_index in splitter.split(np.arange(len(targets))):
            prediction = np.asarray(
                predict_fold(train_index, validation_index, candidate), dtype=float
            )
            fold_scores.append(
                float(np.mean(np.abs(targets[validation_index] - prediction)))
            )
        scores[candidate] = float(np.mean(fold_scores))

    selected = min(candidates, key=lambda candidate: scores[candidate])
    return selected, scores


def regression_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return unclipped regression metrics for one target."""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    if y_true.shape != y_pred.shape:
        raise ValueError("y_true and y_pred must have identical shapes")

    pred_sd = float(np.std(y_pred, ddof=0))
    if pred_sd == 0:
        spearman = np.nan
        calibration_intercept = float(np.mean(y_true))
        calibration_slope = 0.0
    else:
        spearman = float(pd.Series(y_true).corr(pd.Series(y_pred), method="spearman"))
        design = np.column_stack([np.ones(len(y_pred)), y_pred])
        calibration_intercept, calibration_slope = np.linalg.lstsq(
            design, y_true, rcond=None
        )[0]

    return {
        "r2": float(r2_score(y_true, y_pred)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "rmse": float(mean_squared_error(y_true, y_pred) ** 0.5),
        "spearman": spearman,
        "calibration_intercept": float(calibration_intercept),
        "calibration_slope": float(calibration_slope),
    }
```

ensemble_models.py

```
from __future__ import annotations
import numpy as np
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.impute import SimpleImputer

def fit_predict_hgb_multioutput(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
```

```
    *,
    max_leaf_nodes: int,
    min_samples_leaf: int,
    learning_rate: float,
    l2_regularization: float,
    max_iter: int,
    random_state: int,
) -> np.ndarray:
    """Fit one fold-local HistGradientBoostingRegressor per target."""
    targets = np.asarray(y_train, dtype=float)
    if targets.ndim != 2 or not np.isfinite(targets).all():
        raise ValueError("y_train must be a finite two-dimensional matrix")
    if len(x_train) != len(targets):
        raise ValueError("x_train and y_train have different lengths")
    imputer = SimpleImputer(strategy="median", add_indicator=True)
    train_features = imputer.fit_transform(np.asarray(x_train, dtype=float))
    validation_features = imputer.transform(np.asarray(x_validation, dtype=float))
    columns = []
    for target_position in range(targets.shape[1]):
        model = HistGradientBoostingRegressor(
            max_leaf_nodes=max_leaf_nodes,
            min_samples_leaf=min_samples_leaf,
            learning_rate=learning_rate,
            l2_regularization=l2_regularization,
            max_iter=max_iter,
            random_state=random_state + target_position,
        )
        model.fit(train_features, targets[:, target_position])
        columns.append(model.predict(validation_features))
    prediction = np.column_stack(columns)
    if not np.isfinite(prediction).all():
        raise RuntimeError("continuous HGB produced non-finite predictions")
    return prediction
```

model_config.json

```
{
  "base_models": [
    "extra_trees",
    "hgb",
    "ridge_quantile",
    "pls2_quantile"
  ],
  "extra_trees": {
    "n_estimators": 300,
    "max_features": [
      0.33,
      0.6,
      1.0
    ],
    "min_samples_leaf": [
      1,
      2,
      4,
      8
    ],
    "max_depth": null
  },
  "hgb_profiles": [
    {
      "name": "compact",
```

```
      "max_leaf_nodes": 7,
      "min_samples_leaf": 20,
      "learning_rate": 0.05,
      "l2_regularization": 1.0,
      "max_iter": 100
    },
    {
      "name": "balanced",
      "max_leaf_nodes": 15,
      "min_samples_leaf": 20,
      "learning_rate": 0.05,
      "l2_regularization": 1.0,
      "max_iter": 100
    }
  ],
  "ridge_alphas": [
    0.1,
    1.0,
    10.0,
    100.0,
    1000.0
  ],
  "pls_components": [
    2,
    5,
    10,
    15,
    20
  ]
}
```

nnls_stacking.py

```
"""Nonnegative OOF model weights with free target intercepts and fixed shrinkage."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import nnls


@dataclass(frozen=True)
class NNLSStackingFit:
    weights: np.ndarray
    intercept: np.ndarray
    global_weights: np.ndarray
    scope: str
    shrinkage: float

    def predict(self, stack) -> np.ndarray:
        values = np.asarray(stack, dtype=float)
        if values.ndim != 3 or values.shape[1:] != self.weights.shape:
            raise ValueError("prediction tensor must have the fitted targets/models shape")
        if not np.isfinite(values).all():
            raise ValueError("prediction tensor must be finite")
        return np.einsum("ntm,tm->nt", values, self.weights) + self.intercept


def _solve(centered_stack, centered_y, scales) -> np.ndarray:
```

```
    design = (centered_stack / scales[None, :, None]).reshape(-1, centered_stack.shape[2])
    response = (centered_y / scales[None, :]).reshape(-1)
    if not np.any(design):
        return np.zeros(design.shape[1], dtype=float)
    weights, _ = nnls(design, response, maxiter=max(100, design.shape[1] * 30))
    return weights


def fit_nnls_stacking(stack, y, *, groups=None, scope="shared", shrinkage=.5) -> NNLSStackingFit:
    """Fit on outer-training inner-OOF predictions and their training labels only.

    Weights are nonnegative but need not sum to one. Target centering removes
    intercepts before NNLS, so an intercept is free to be negative afterward.
    Standardizing each training target gives shared/group fits equal target units.
    """
    predictions, truth = np.asarray(stack, dtype=float), np.asarray(y, dtype=float)
    if (predictions.ndim != 3 or min(predictions.shape) < 1
            or predictions.shape[0] < 3 or truth.shape != predictions.shape[:2]
            or not np.isfinite(predictions).all() or not np.isfinite(truth).all()):
        raise ValueError("finite samples-by-targets-by-models predictions must align with y")
    if scope not in {"shared", "group", "target"}:
        raise ValueError("scope must be shared, group or target")
    if not np.isfinite(shrinkage) or not 0 <= shrinkage <= 1:
        raise ValueError("shrinkage must be in [0, 1]")
    if scope == "group" and (groups is None or len(groups) != truth.shape[1]):
        raise ValueError("group labels must cover all registered targets")
    means = predictions.mean(axis=0)
    target_means = truth.mean(axis=0)
    centered, y_centered = predictions - means, truth - target_means
    scales = np.std(truth, axis=0)
    scales = np.where(scales > 1e-12, scales, 1.)
    global_weights = _solve(centered, y_centered, scales)
    weights = np.tile(global_weights, (truth.shape[1], 1))
    if scope != "shared":
        labels = np.arange(truth.shape[1]) if scope == "target" else np.asarray(groups, dtype=str)
        for label in dict.fromkeys(labels.tolist()):
            positions = np.flatnonzero(labels == label)
            local = _solve(centered[:, positions], y_centered[:, positions], scales[positions])
            weights[positions] = (1. - shrinkage) * local + shrinkage * global_weights
    intercept = target_means - np.einsum("tm,tm->t", means, weights)
    return NNLSStackingFit(weights.copy(), intercept.copy(), global_weights.copy(),
                           str(scope), 1. if scope == "shared" else float(shrinkage))
```

plot_final.R

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(ggplot2)
  library(dplyr)
  library(tidyr)
  library(readr)
  library(patchwork)
})

source("plot_style.R")

data_dir <- "../数据与结果"
figure_dir <- Sys.getenv(
  "Q1_FIGURE_DIR",
  unset = "../图片"
```

```
)
dir.create(figure_dir, recursive = TRUE, showWarnings = FALSE)
save_png <- save_paper
save_paper <- function(plot, file, width = 6, height = 4, dpi = 300) {
  save_png(plot, file, width, height, dpi)
  ggsave(sub("\\.png$", ".svg", file), plot, device = svglite::svglite,
         width = width, height = height, units = "in", bg = "white")
}

# 论文图文件内不写整图标题。图题只保存在 Q1_图清单.csv，
# 后续由 DOCX 在图下方统一排版。面板标识可用 (a)(b)，分面标签无底色并加粗。

group_labels <- c(
  "Aroma" = "香气",
  "Flavour-core" = "风味主体",
  "Taste-other" = "其他味觉",
  "Palate" = "口感",
  "Overall" = "总体"
)
group_colors <- c(
  "Aroma" = "#3B75AF",
  "Flavour-core" = "#EF8636",
  "Taste-other" = "#55A868",
  "Palate" = "#C44E52",
  "Overall" = "#8172B3"
)
tier_colors <- c("高" = "#2563A6", "中" = "#E39C34", "低" = "#8C8C8C")

# 原始字段代码继续留在数据层；论文图的刻度与图例使用中文展示名。
target_labels_zh <- c(
  "A_malt_all" = "麦芽香气-总体",
  "A_malt_grain" = "麦芽香气-谷物",
  "A_malt_bread" = "麦芽香气-面包",
  "A_malt_cara" = "麦芽香气-焦糖",
  "A_malt_burn" = "麦芽香气-烟熏",
  "A_hops_all" = "酒花香气-总体",
  "A_hops_citrus" = "酒花香气-柑橘",
  "A_hops_tropical" = "酒花香气-热带水果",
  "A_hops_noble" = "酒花香气-青草",
  "A_hops_woody" = "酒花香气-木质",
  "A_esters_all" = "酯类香气-总体",
  "A_esters_ethac" = "酯类香气-指甲油",
  "A_esters_isoaa" = "酯类香气-香蕉",
  "A_esters_flower" = "酯类香气-花香",
  "A_esters_fruity" = "酯类香气-果香",
  "F_malt_all" = "麦芽风味-总体",
  "F_malt_grain" = "麦芽风味-谷物",
  "F_malt_bread" = "麦芽风味-面包",
  "F_malt_cara" = "麦芽风味-焦糖",
  "F_malt_burn" = "麦芽风味-烟熏",
  "F_hops_all" = "酒花风味-总体",
  "F_hops_citrus" = "酒花风味-柑橘",
  "F_hops_tropical" = "酒花风味-热带水果",
  "F_hops_noble" = "酒花风味-青草",
  "F_hops_woody" = "酒花风味-木质",
  "F_esters_all" = "酯类风味-总体",
  "F_esters_ethac" = "酯类风味-指甲油",
  "F_esters_isoaa" = "酯类风味-香蕉",
  "F_esters_flower" = "酯类风味-花香",
  "F_esters_fruity" = "酯类风味-果香",
  "acidity" = "酸度",
```

```
  "bitternes" = "苦味",
  "sweetness" = "甜味",
  "X4vg" = "4VG",
  "diacetyl" = "双乙酰",
  "dms" = "二甲基硫（DMS）",
  "metallic" = "金属味",
  "stale_hops" = "陈旧酒花味",
  "t2n" = "反式-2-壬烯醛",
  "orange" = "橙香",
  "coriander" = "芫荽",
  "clove" = "丁香",
  "lactic" = "乳酸味",
  "acetic" = "乙酸味",
  "barnyard" = "畜棚味",
  "alcohol" = "酒精感",
  "aftertaste" = "回味",
  "body" = "酒体",
  "co2" = "碳酸感",
  "overall" = "总体喜好"
)

style_labels_zh <- c(
  "Amber" = "琥珀艾尔", "Blond" = "金色艾尔",
  "Brett/cofermented" = "Brett/混合发酵", "Brown" = "棕色艾尔",
  "Brut" = "Brut（干型）", "Christmas" = "圣诞啤酒",
  "Dubbel" = "双料啤酒", "Faro" = "法柔",
  "Flanders old brown" = "佛兰德老棕艾尔", "Fruitbeer" = "水果啤酒",
  "Hoppy" = "酒花型", "Kriek" = "克里克（樱桃）",
  "Lambic" = "兰比克", "Low/No alcohol" = "低/无醇",
  "Pils/Lager" = "皮尔森/拉格", "Saison" = "赛松",
  "Scotch" = "苏格兰艾尔", "Stout/Porter" = "世涛/波特",
  "Strong ale" = "强艾尔", "Tripel" = "三料啤酒",
  "West Flanders ale" = "西佛兰德艾尔", "Wheat" = "小麦啤酒"
)

panel_header_theme <- theme(
  plot.subtitle = element_text(
    colour = "black", face = "bold", size = 9.5, hjust = 0,
    margin = margin(b = 5)
  )
)

# 图 1：相同重复划分下比较基线、三种融合方法和最终堆叠模型。
comparison <- read_csv(file.path(data_dir, "model_comparison.csv"),
                       show_col_types = FALSE)
model_levels <- c("ExtraTrees 基线", "连续凸融合", "连续—有序融合",
                  "逐目标收缩融合", "感官组收缩 NNLS")
model_colors <- c(
  "ExtraTrees 基线" = "#88A9D3",
  "连续凸融合" = "#67A995",
  "连续—有序融合" = "#D4A35B",
  "逐目标收缩融合" = "#9A82B8",
  "感官组收缩 NNLS" = "#1F5A94"
)
model_outlines <- c(
  "ExtraTrees 基线" = "#4F719B",
  "连续凸融合" = "#3F7568",
  "连续—有序融合" = "#8C6735",
  "逐目标收缩融合" = "#66527F",
  "感官组收缩 NNLS" = "#123A65"
)
```

```
comparison <- comparison %>%
  mutate(display_name = factor(display_name, levels = model_levels))
focus_summary <- comparison %>%
  filter(display_name == "感官组收缩NNLS") %>%
  summarise(
    r2 = mean(macro_r2),
    mae = mean(macro_mae),
    r2_label_y = mean(macro_r2),
    mae_label_y = mean(macro_mae)
  ) %>%
  mutate(display_name = factor("感官组收缩NNLS", levels = model_levels))

p1a <- ggplot(comparison, aes(display_name, macro_r2,
                              fill = display_name, colour = display_name)) +
  geom_boxplot(width = 0.56, linewidth = 0.38, outlier.shape = NA, alpha = 0.88) +
  geom_point(position = position_jitter(width = 0.10, height = 0,
                                        seed = 20261023),
             shape = 21, size = 1.55, stroke = 0.28) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.35,
               stroke = 0.45, fill = "white", colour = "black") +
  geom_text(
    data = focus_summary,
    aes(x = display_name, y = r2_label_y, label = sprintf("%.3f", r2)),
    inherit.aes = FALSE, colour = "#123A65", size = 3.0,
    fontface = "bold", vjust = 0.5, hjust = 0, nudge_x = 0.30
  ) +
  scale_fill_manual(values = model_colors) +
  scale_colour_manual(values = model_outlines) +
  scale_y_continuous(expand = expansion(mult = c(0.08, 0.16))) +
  scale_x_discrete(labels = c("ExtraTrees\n基线", "连续\n凸融合", "连续—有序\n融合", "逐目标\n收缩融合", "感
官组收缩\nNNLS"), expand = expansion(add = c(.6,.95))) +
  labs(x = NULL, y = "宏平均 R²", subtitle = "（a）解释度") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5, size = 8))

p1b <- ggplot(comparison, aes(display_name, macro_mae,
                              fill = display_name, colour = display_name)) +
  geom_boxplot(width = 0.56, linewidth = 0.38, outlier.shape = NA, alpha = 0.88) +
  geom_point(position = position_jitter(width = 0.10, height = 0,
                                        seed = 20261023),
             shape = 21, size = 1.55, stroke = 0.28) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.35,
               stroke = 0.45, fill = "white", colour = "black") +
  geom_text(
    data = focus_summary,
    aes(x = display_name, y = mae_label_y, label = sprintf("%.3f", mae)),
    inherit.aes = FALSE, colour = "#123A65", size = 3.0,
    fontface = "bold", vjust = 0.5, hjust = 0, nudge_x = 0.30
  ) +
  scale_fill_manual(values = model_colors) +
  scale_colour_manual(values = model_outlines) +
  scale_y_continuous(expand = expansion(mult = c(0.14, 0.08))) +
  scale_x_discrete(labels = c("ExtraTrees\n基线", "连续\n凸融合", "连续—有序\n融合", "逐目标\n收缩融合", "感
官组收缩\nNNLS"), expand = expansion(add = c(.6,.95))) +
  labs(x = NULL, y = "宏平均 MAE（z 分数）", subtitle = "（b）绝对误差") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5, size = 8))

p1 <- (p1a | p1b)
```

```
save_paper(p1, file.path(figure_dir, "图1_模型对比.png"),
           width = 8.2, height = 4.35)

# 图2：逐目标 OOF 性能图谱。三栏分别承载香气、风味和其余目标，
# 每行用同一条短连线比较 R² 与 Spearman，避免 50 行双栏图的视觉拥挤。
target_metrics <- read_csv(file.path(data_dir, "target_metrics_final.csv"),
                           show_col_types = FALSE)
target_metrics <- target_metrics %>%
  mutate(
    panel_group = case_when(
      target_group == "Aroma" ~ "香气",
      target_group == "Flavour-core" ~ "风味",
      TRUE ~ "味觉·口感·总体"
    ),
    detail_order = case_when(
      target_group == "Taste-other" ~ 1L,
      target_group == "Palate" ~ 2L,
      target_group == "Overall" ~ 3L,
      TRUE ~ 1L
    )
  )

target_panel_plot <- function(data, panel_title, y_text_size = 6.5,
                              separators = numeric()) {
  panel_data <- data %>%
    arrange(detail_order, r2) %>%
    mutate(
      target_label = unname(target_labels_zh[target_code]),
      target_label = factor(target_label, levels = target_label)
    )
  metric_long <- bind_rows(
    panel_data %>% transmute(target_label, value = r2, metric = "R²"),
    panel_data %>% transmute(target_label, value = spearman,
                             metric = "Spearman")
  )

  plot <- ggplot(panel_data, aes(y = target_label)) +
    geom_vline(xintercept = c(0.2, 0.4, 0.6, 0.8),
               colour = "#E5E9EF", linewidth = 0.34) +
    geom_segment(aes(x = r2, xend = spearman, yend = target_label),
                 colour = "#C9D0DA", linewidth = 0.62, lineend = "round") +
    geom_point(data = metric_long,
               aes(x = value, shape = metric, fill = metric, colour = metric),
               size = 2.05, stroke = 0.72) +
    scale_shape_manual(values = c("R²" = 21, "Spearman" = 23)) +
    scale_fill_manual(values = c("R²" = "#1F5A94", "Spearman" = "white")) +
    scale_colour_manual(values = c("R²" = "#123A65",
                                   "Spearman" = "#D06B32")) +
    scale_x_continuous(limits = c(0, 0.9), breaks = seq(0, 0.8, 0.2),
                       expand = expansion(mult = c(0.015, 0.02))) +
    labs(x = "OOF 性能", y = NULL, subtitle = panel_title) +
    theme_mcup(base_size = 8.6, legend = "right") +
    panel_header_theme +
    theme(
      axis.text.y = element_text(size = y_text_size),
      legend.position = "bottom",
      legend.direction = "horizontal",
      legend.text = element_text(size = 7.2),
      plot.margin = margin(4, 5, 3, 4)
    )
```

```
  if (length(separators) > 0) {
    plot <- plot + geom_hline(
      yintercept = separators, colour = "#9AA3AE",
      linewidth = 0.42, linetype = "dotted"
    )
  }
  plot
}

p2a <- target_panel_plot(
  filter(target_metrics, panel_group == "香气"),
  "（a）香气目标"
)
p2b <- target_panel_plot(
  filter(target_metrics, panel_group == "风味"),
  "（b）风味目标"
)
p2c <- target_panel_plot(
  filter(target_metrics, panel_group == "味觉·口感·总体"),
  "（c）味觉、口感与总体", y_text_size = 6.0,
  separators = c(15.5, 19.5)
)

p2 <- (p2a | p2b | p2c) +
  plot_layout(widths = c(1, 1, 1), guides = "collect") &
  theme(legend.position = "bottom")
save_paper(p2, file.path(figure_dir, "图2_逐目标预测性能.png"),
           width = 11.2, height = 6.3)

# 图3：高/中/低各保留一个预登记代表目标，减少重复子图。
representative <- read_csv(file.path(data_dir, "representative_oof.csv.gz"),
                           show_col_types = FALSE)
representative3 <- representative %>%
  filter(target_code %in% c("acidity", "X4vg", "stale_hops")) %>%
  left_join(target_metrics %>% select(target_code, r2, mae), by = "target_code") %>%
  mutate(
    facet_label_clean = paste0(unname(target_labels_zh[target_code]),
                               "   R²=", sprintf("%.2f", r2),
                               "，MAE=", sprintf("%.2f", mae)),
    facet_label_clean = factor(
      facet_label_clean,
      levels = unique(facet_label_clean[order(match(target_code, c("acidity", "X4vg", "stale_hops")))])
    )
  )

p3 <- ggplot(representative3, aes(y_true, y_pred_mean)) +
  geom_abline(slope = 1, intercept = 0, linetype = 2, colour = "#555555",
              linewidth = 0.48) +
  geom_smooth(aes(colour = tier, fill = tier), method = "lm", formula = y ~ x,
              se = TRUE, linewidth = 0.68, alpha = 0.12) +
  geom_point(aes(colour = tier), size = 1.25, alpha = 0.52) +
  facet_wrap(~facet_label_clean, nrow = 1) +
  scale_colour_manual(values = tier_colors) +
  scale_fill_manual(values = tier_colors) +
  labs(x = "观测值（z 分数）", y = "OOF 预测值（z 分数）") +
  theme_mcup(base_size = 9.5, legend = "none") +
  theme(strip.background = element_blank(),
        strip.text = element_text(face = "bold", size = 8.5,
                                  margin = margin(1, 0, 4, 0)))
save_paper(p3, file.path(figure_dir, "图3_OOF观测预测.png"),
           width = 8.8, height = 4.2)
```

```
# 图 4：将风格稳健性与 0–6 输出口径组成横版双联图。
style_metrics <- read_csv(file.path(data_dir, "style_metrics_final.csv"),
                          show_col_types = FALSE) %>%
  arrange(mae_all_targets) %>%
  mutate(tasting_category_fine = factor(tasting_category_fine,
                                        levels = tasting_category_fine))
style_axis_labels <- setNames(
  paste0(unname(style_labels_zh[as.character(style_metrics$tasting_category_fine)]),
         "  (", style_metrics$n_beers, ")"),
  as.character(style_metrics$tasting_category_fine)
)
mean_oof_mae <- mean(target_metrics$mae)
p4a <- ggplot(style_metrics,
              aes(mae_all_targets, tasting_category_fine,
                  colour = mae_all_targets)) +
  geom_vline(xintercept = mean_oof_mae, linetype = 2,
             colour = "#222222", linewidth = 0.48) +
  geom_segment(aes(x = baseline_mae, xend = mae_all_targets,
                   yend = tasting_category_fine), colour = "grey78", linewidth = 0.45) +
  geom_point(aes(x = baseline_mae, shape = "ExtraTrees 基线"),
             colour = "#8A929C", size = 2.1, stroke = 0.7) +
  geom_point(aes(size = n_beers, shape = "最终模型"), alpha = 0.93) +
  scale_shape_manual(values = c("ExtraTrees 基线" = 1, "最终模型" = 16),
                     breaks = c("ExtraTrees 基线", "最终模型"),
                     labels = c("空心：ExtraTrees 基线", "实心：最终模型（按 MAE 着色）"),
                     name = NULL) +
  guides(shape = guide_legend(ncol = 1, override.aes = list(
    colour = c("#8A929C", "#2C7FB8"), size = c(2.3, 2.8)))) +
  scale_colour_gradient(
    low = "#2C7FB8", high = "#C43C4E",
    name = "颜色：MAE（蓝低，红高）",
    guide = guide_colourbar(
      direction = "horizontal", title.position = "top",
      barwidth = unit(3.6, "cm"), barheight = unit(0.18, "cm")
    )
  ) +
  scale_size_continuous(range = c(2.0, 5.0), guide = "none") +
  scale_y_discrete(labels = style_axis_labels) +
  scale_x_continuous(expand = expansion(mult = c(0.06, 0.06))) +
  labs(x = sprintf("MAE（z 分数；总体 %.3f）", mean_oof_mae), y = NULL,
       subtitle = "（a）平均折外预测的风格误差") +
  theme_mcup(base_size = 8.8, legend = "none") +
  panel_header_theme +
  theme(axis.text.y = element_text(size = 7.2),
        legend.position = "bottom",
        legend.direction = "horizontal",
        legend.title = element_text(face = "bold", size = 7.3),
        legend.text = element_text(size = 6.8))

mapping <- read_csv(file.path(data_dir, "scale_mapping_example.csv"),
                    show_col_types = FALSE) %>%
  mutate(
    target_display = recode(
      target_code,
      "A_malt_all" = "总体麦芽香气",
      "F_esters_flower" = "花香型酯类风味",
      "acidity" = "酸度",
      "alcohol" = "酒精感"
    )
  )
```

```
mapping_colors <- c(
  "总体麦芽香气" = "#2C7FB8",
  "花香型酯类风味" = "#D1495B",
  "酒精感" = "#55A868",
  "酸度" = "#8172B3"
)
mapping_linetypes <- c(
  "总体麦芽香气" = "solid",
  "花香型酯类风味" = "longdash",
  "酒精感" = "dotdash",
  "酸度" = "twodash"
)
p4b <- ggplot(mapping,
              aes(z_score, grade_0_6,
                  colour = target_display, linetype = target_display)) +
  geom_step(linewidth = 0.82, direction = "hv") +
  scale_colour_manual(values = mapping_colors, name = NULL) +
  scale_linetype_manual(values = mapping_linetypes, name = NULL) +
  scale_y_continuous(breaks = 0:6, limits = c(0, 6)) +
  labs(x = "模型预测的聚合 z 分数", y = "0–6 相对强度等级",
       subtitle = "（b）训练折经验分布的单调映射") +
  guides(
    colour = guide_legend(nrow = 2, byrow = TRUE),
    linetype = guide_legend(nrow = 2, byrow = TRUE)
  ) +
  theme_mcup(base_size = 8.8, legend = "right") +
  panel_header_theme +
  theme(
    legend.position = "bottom",
    legend.direction = "horizontal",
    legend.text = element_text(size = 6.8),
    legend.key.width = unit(1.15, "cm"),
    legend.spacing.x = unit(0.12, "cm")
  )

hard <- read_csv(file.path(data_dir, "preregistered_hard_targets.csv"),
                 show_col_types = FALSE) %>%
  arrange(delta_r2) %>%
  mutate(target_label = factor(unname(target_labels_zh[target_code]),
                               levels = unname(target_labels_zh[target_code])),
         change = ifelse(delta_r2 >= 0, "改善", "下降"))
p4hard <- ggplot(hard, aes(delta_r2, target_label, colour = change)) +
  geom_vline(xintercept = 0, linetype = 2, linewidth = 0.45, colour = "#555555") +
  geom_segment(aes(x = 0, xend = delta_r2, yend = target_label),
               linewidth = 1.0, alpha = .7) +
  geom_point(aes(shape = change), size = 2.6) +
  scale_colour_manual(values = c("改善" = "#1F5A94", "下降" = "#C57C39"), name = NULL) +
  scale_shape_manual(values = c("改善" = 16, "下降" = 17), name = NULL) +
  scale_x_continuous(expand = expansion(mult = c(.15,.15))) +
  labs(x = "相对基线的 R² 变化", y = NULL, subtitle = "（b）预登记难目标的改善与退化") +
  theme_mcup(base_size = 8.8, legend = "right") + panel_header_theme +
  theme(legend.position = "bottom", axis.text.y = element_text(size = 7.5))
p4 <- (p4a | p4hard) +
  plot_layout(widths = c(1.12, 0.88))
save_paper(p4, file.path(figure_dir, "图4_风格与难目标稳健性.png"),
           width = 10.5, height = 5.9)

# 附图 S1：保留诊断价值，但不占用正文图数。
p_s1 <- ggplot(representative3, aes(y_pred_mean, residual)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "#555555", linewidth = 0.45) +
  geom_point(aes(colour = tier), size = 1.30, alpha = 0.58) +
```

```
  geom_smooth(method = "loess", formula = y ~ x, se = TRUE,
              colour = "#1F1F1F", fill = "#C9C9C9", linewidth = 0.62,
              alpha = 0.24) +
  facet_wrap(~facet_label_clean, nrow = 1) +
  scale_colour_manual(values = tier_colors) +
  labs(x = "OOF 预测值（z 分数）", y = "残差（观测−预测）", subtitle = "（a）代表目标的残差诊断") +
  theme_mcup(base_size = 9.5, legend = "none") +
  panel_header_theme + theme(strip.background = element_blank(),
        strip.text = element_text(face = "bold", size = 8.5,
                                  margin = margin(1, 0, 4, 0)))
p_s1_combined <- p_s1 / p4b + plot_layout(heights = c(1, 0.85))
save_paper(p_s1_combined, file.path(figure_dir, "附图 S1_残差诊断与量表映射.png"),
           width = 9.5, height = 7.0)

manifest <- tibble::tribble(
  ~figure_no, ~role, ~path, ~caption, ~explanation,
  "1", "main", file.path(figure_dir, "图 1_模型对比.png"),
  "图 1 基线、融合与感官组收缩 NNLS 的重复折外性能比较",
  "散点表示 10 次重复，箱体表示重复间分布；两个面板分别衡量解释度和绝对误差，纵轴为局部范围。",
  "2", "main", file.path(figure_dir, "图 2_逐目标预测性能.png"),
  "图 2 最终模型 50 个感官目标的 OOF R² 与 Spearman 性能图谱",
  "三栏按感官家族展示全部目标；同一目标的 R² 与 Spearman 共同刻画可预测性异质性。",
  "3", "main", file.path(figure_dir, "图 3_OOF 观测预测.png"),
  "图 3 高、中、低可预测感官目标的 OOF 观测—预测关系",
  "酸度、4VG 和陈旧酒花味为此前固定的代表目标；虚线为理想预测，实线及阴影为 OLS 拟合及其均值置信带。",
  "4", "main", file.path(figure_dir, "图 4_风格与难目标稳健性.png"),
  "图 4 最终模型的风格误差及预登记难目标的稳健性检验",
  "左图为 mean-OOF 风格误差，空心点为基线、实心点为最终模型，括号和点大小表示样本量；右图为 10 次重复平均 R² 差值。",
  "S1", "supplementary", file.path(figure_dir, "附图 S1_残差诊断与量表映射.png"),
  "附图 S1 折外残差诊断与训练折经验分布的相对量表映射",
  "上图 LOESS 用于检查系统偏差；下图为固定训练折的 CDF 示例，0–6 仅表示相对强度，图例置于下方。"
)
write_csv(manifest, file.path(figure_dir, "Q1_图清单.csv"))
cat("Generated 4 main figures and 1 supplementary figure at", figure_dir, "\n")
```

plot_style.R

```
# 论文出图统一样式（R / ggplot2）
#
# 视觉规范来源：40 篇 M 杯优秀论文 615 张图表实测 + 逐页目检（docs/M 杯优秀论文特点、题型特点.md §4.5）
#   - 白底、黑色细边框、无网格、图例内嵌且无边框、3:2 横版
#   - 中文宋体 + 拉丁 Times 风格（SimSun 自带）
#   - 训练曲线四色；PR 总曲线深蓝加粗；混淆矩阵 Blues；热力图 hot
# 项目强制规则：图片文件内不放整图标题/图号，图题在论文中置于图下；
# 分面标签无灰色底框并加粗，轴标题加粗，主信息需有明确视觉强调。
#
# 用法：
#   source("scripts/plot_style.R")
#   p <- ggplot(...) + theme_mcup()
#   save_paper(p, "figures/fig1.png")
#   plot_confusion_matrix(cm, c("dent","hole","rusty"), file = "figures/cm.png")
#   plot_pr_curves(curves, file = "figures/pr.png")
#   plot_corr_heatmap(cor_mat, file = "figures/corr.png")
#   plot_feature_importance(names, values, file = "figures/imp.png")
suppressPackageStartupMessages({
  library(ggplot2)
  library(ragg)
})

FONT_PAPER <- "SimSun"
FONT_SANS  <- "SimHei"
```

```
CYCLE <- c("#1f77b4", "#ff7f0e", "#2ca02c", "#d62728")
NAVY  <- "#00007f"

# 逐条重要性条用的冷暖色（近似 matplotlib Spectral，自上而下红→蓝）
SPECTRAL <- c("#D53E4F", "#F46D43", "#FDAE61", "#FEE08B",
              "#E6F598", "#ABDDA4", "#66C2A5", "#3288BD")
spectral_n <- function(n) colorRampPalette(SPECTRAL)(n)

# 紧凑、干净的 M 杯主题
# legend: "br"=右下、"tr"=右上、"none"=不要图例
theme_mcup <- function(mode = c("paper", "sans"), base_size = 10, legend = "br",
                       title_family = NULL) {
  mode <- match.arg(mode)
  fam <- if (mode == "paper") FONT_PAPER else FONT_SANS
  lg <- switch(legend,
    br = list(pos = c(0.985, 0.02), just = c(1, 0)),
    tr = list(pos = c(0.985, 0.985), just = c(1, 1)),
    bl = list(pos = c(0.015, 0.02), just = c(0, 0)),
    right = list(pos = "right"),
    none = NULL,
    stop("legend 必须是 br / tr / bl / right / none")
  )
  th <- theme_bw(base_size = base_size, base_family = fam) +
    theme(
      panel.grid = element_blank(),
      panel.border = element_rect(colour = "black", linewidth = 0.5),
      panel.background = element_rect(fill = "white", colour = NA),
      plot.background = element_rect(fill = "white", colour = NA),
      axis.text = element_text(colour = "black", size = base_size - 1),
      axis.title = element_text(colour = "black", size = base_size - 0.5,
                                face = "bold"),
      axis.ticks = element_line(colour = "black", linewidth = 0.35),
      axis.ticks.length = unit(1.6, "pt"),
      plot.title = element_blank(),
      plot.tag = element_text(colour = "black", face = "bold",
                              size = base_size),
      strip.background = element_blank(),
      strip.text = element_text(colour = "black", face = "bold",
                                size = base_size - 0.5,
                                margin = margin(2, 1, 3, 1)),
      plot.margin = margin(4, 5, 3, 3),
      legend.background = element_blank(),
      legend.box.background = element_blank(),
      legend.key = element_blank(),
      legend.title = element_blank(),
      legend.text = element_text(size = base_size - 1.5),
      legend.key.size = unit(0.82, "lines"),
      legend.key.width = unit(1.15, "lines"),
      legend.spacing.y = unit(0.02, "cm"),
      legend.margin = margin(1.5, 2.5, 1.5, 2.5)
    )
  if (is.null(lg)) {
    th + theme(legend.position = "none")
  } else if (!is.null(lg$pos) && length(lg$pos) == 1 && lg$pos == "right") {
    th + theme(legend.position = "right",
               legend.justification = "center",
               legend.margin = margin(0, 0, 0, 4),
               legend.key.height = unit(0.6, "lines"))
  } else {
    th + theme(legend.position = "inside",
```

```
              legend.position.inside = lg$pos,
              legend.justification = lg$just)
  }
}

# 保存 300dpi PNG
save_paper <- function(plot, file, width = 6, height = 4, dpi = 300) {
  dir.create(dirname(file), showWarnings = FALSE, recursive = TRUE)
  agg_png(file, width = width, height = height, units = "in", res = dpi, background = "white")
  on.exit(invisible(dev.off()), add = TRUE)
  print(plot)
  invisible(file)
}

# 混淆矩阵：Blues + 每格数值（深色格白字），短类名不旋转、无留白
plot_confusion_matrix <- function(cm, classes, file = NULL,
                                  normalize = FALSE, width = 4.2, height = 3.6) {
  stopifnot(is.matrix(cm), nrow(cm) == ncol(cm), length(classes) == nrow(cm))
  df <- expand.grid(pred = classes, true = classes)
  df$n <- as.vector(t(cm))
  if (normalize) {
    df$value <- df$n / ave(df$n, df$true, FUN = sum)
    lab <- sprintf("%.2f", df$value)
  } else {
    df$value <- df$n
    lab <- as.character(df$n)
  }
  thresh <- max(df$value) / 2
  df$txt <- ifelse(df$value > thresh, "white", "black")
  angle <- if (max(nchar(classes)) <= 7) 0 else 35
  p <- ggplot(df, aes(pred, true, fill = value)) +
    geom_tile(colour = "white", linewidth = 0.45) +
    geom_text(aes(label = lab, colour = txt), size = 3.1) +
    scale_colour_identity() +
    scale_fill_gradient(low = "#F7FBFF", high = "#08306B",
                        guide = guide_colourbar(barwidth = unit(0.32, "cm"),
                                                barheight = unit(2.3, "cm"),
                                                frame.colour = NA)) +
    scale_x_discrete(expand = c(0, 0)) +
    scale_y_discrete(expand = c(0, 0), limits = rev(classes)) +
    labs(x = "预测标签", y = "真实标签") +
    coord_fixed() +
    theme_mcup(legend = "right") +
    theme(axis.text.x = element_text(angle = angle, hjust = if (angle == 0) 0.5 else 1),
          axis.title.x = element_text(margin = margin(t = 6)),
          axis.title.y = element_text(margin = margin(r = 6)))
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# PR 曲线：各类细线 + 总曲线深蓝加粗，图例带 AP，无边框
plot_pr_curves <- function(curves, file = NULL, width = 5, height = 4) {
  dfs <- lapply(names(curves), function(nm) {
    d <- curves[[nm]]
    data.frame(recall = d$recall, precision = d$precision,
               cls = nm, ap = d$ap, stringsAsFactors = FALSE)
  })
  df <- do.call(rbind, dfs)
  df$is_all <- tolower(df$cls) %in% c("all", "all classes", "micro", "总体")
  others <- setdiff(unique(df$cls[!df$is_all]), character(0))
  lev <- c(others, if (any(df$is_all)) "all classes")
```

```
  lab <- vapply(lev, function(k) {
    ap <- df$ap[df$cls == k][1]
    if (is.na(ap)) ap <- df$ap[df$is_all][1]
    if (k == "all classes") sprintf("all classes %.3f mAP@0.5", ap) else sprintf("%s %.3f", k, ap)
  }, character(1))
  df$grp <- factor(ifelse(df$is_all, "all classes", df$cls), levels = lev, labels = lab)
  col_map <- setNames(c(CYCLE[seq_along(others)], NAVY), lab)
  lw_map <- setNames(ifelse(lev == "all classes", 1.15, 0.55), lab)
  p <- ggplot(df, aes(recall, precision, colour = grp, linewidth = grp)) +
    geom_line() +
    scale_colour_manual(values = col_map) +
    scale_linewidth_manual(values = lw_map, guide = "none") +
    coord_cartesian(xlim = c(0, 1), ylim = c(0, 1.01), expand = FALSE) +
    labs(x = "Recall", y = "Precision", title = "Precision-Recall Curve") +
    theme_mcup(legend = "bl")
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# 相关性热力图：hot（负值深、正值浅）+ 全格数值
plot_corr_heatmap <- function(mat, file = NULL, width = 5.2, height = 4.6) {
  stopifnot(is.matrix(mat), nrow(mat) == ncol(mat))
  nm <- rownames(mat); if (is.null(nm)) nm <- paste0("V", seq_len(nrow(mat)))
  df <- expand.grid(var2 = nm, var1 = nm)
  df$value <- as.vector(t(mat))
  df$txt <- ifelse(df$value < -0.2, "white", "black")
  p <- ggplot(df, aes(var2, var1, fill = value)) +
    geom_tile() +
    geom_text(aes(label = sprintf("%.2f", value), colour = txt), size = 2.4) +
    scale_colour_identity() +
    scale_fill_gradientn(colours = c("black", "red", "orange", "yellow", "white"),
                         limits = c(-1, 1),
                         guide = guide_colourbar(barwidth = unit(0.3, "cm"),
                                                 barheight = unit(2.6, "cm"),
                                                 frame.colour = NA)) +
    scale_x_discrete(expand = c(0, 0)) +
    scale_y_discrete(expand = c(0, 0), limits = rev(nm)) +
    labs(x = NULL, y = NULL, title = "相关性热力图") +
    coord_fixed() +
    theme_mcup(legend = "right") +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
  if (!is.null(file)) save_paper(p, file, width = width, height = height)
  p
}

# 特征重要性：横向降序、逐条冷暖色、末端数值
plot_feature_importance <- function(names, values, file = NULL,
                                    width = 6, height = NULL, top_n = 15) {
  ord <- order(values, decreasing = TRUE)[seq_len(min(top_n, length(values)))]
  df <- data.frame(name = factor(names[ord], levels = rev(names[ord])),
                   value = values[ord])
  df$col <- factor(seq_len(nrow(df)))
  h <- if (is.null(height)) max(3.0, 0.30 * nrow(df) + 0.9) else height
  p <- ggplot(df, aes(value, name, fill = col)) +
    geom_col(width = 0.68) +
    geom_text(aes(label = sprintf("%.1f", value)), hjust = -0.15, size = 2.5) +
    scale_fill_manual(values = spectral_n(nrow(df)), guide = "none") +
    scale_x_continuous(expand = expansion(mult = c(0, 0.10))) +
    labs(x = "Relative Importance", y = NULL, title = "特征重要性") +
    theme_mcup(legend = "none") +
    theme(axis.text.y = element_text(size = 8.5))
```

```
  if (!is.null(file)) save_paper(p, file, width = width, height = h)
  p
}
```

q1_metrics.py

```
"""Reusable continuous and ordered-proxy metrics for Q1 predictions."""

from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.metrics import (
    cohen_kappa_score,
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)


N_LEVELS = 7
METRIC_COLUMNS = [
    "r2", "mae", "rmse", "spearman", "qwk_proxy", "acc_proxy",
    "acc_pm1_proxy", "level_mae_proxy", "level_spearman_proxy",
]


def _safe_spearman(first: np.ndarray, second: np.ndarray) -> float:
    first_array = np.asarray(first, dtype=float)
    second_array = np.asarray(second, dtype=float)
    if first_array.size < 2 or second_array.size < 2:
        return float("nan")
    if np.ptp(first_array) == 0.0 or np.ptp(second_array) == 0.0:
        return float("nan")
    return float(spearmanr(first_array, second_array).statistic)


def _nanmean_without_warning(values: list[float]) -> float:
    array = np.asarray(values, dtype=float)
    return float("nan") if np.isnan(array).all() else float(np.nanmean(array))


def quantile_levels(
    values: np.ndarray,
    reference: np.ndarray,
    n_levels: int = N_LEVELS,
) -> np.ndarray:
    """Map continuous values to integer levels using reference quantiles."""
    value_array = np.asarray(values, dtype=float)
    reference_array = np.asarray(reference, dtype=float)
    if value_array.ndim != 1 or reference_array.ndim != 1:
        raise ValueError("values and reference must both be one-dimensional")
    if not np.isfinite(value_array).all() or not np.isfinite(reference_array).all():
        raise ValueError("values and reference must be finite")
    if n_levels < 2:
        raise ValueError("n_levels must be at least two")
    quantiles = np.linspace(0.0, 1.0, n_levels + 1)[1:-1]
    edges = np.unique(np.quantile(reference_array, quantiles))
    return np.digitize(value_array, edges).astype(int)
```

```
def ordered_proxy_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return seven-bin seven-bin diagnostic metrics for one target."""
    true_levels = quantile_levels(y_true, y_true)
    predicted_levels = quantile_levels(y_pred, y_pred)
    difference = np.abs(true_levels - predicted_levels)
    level_spearman = _safe_spearman(true_levels, predicted_levels)
    return {
        "qwk_proxy": float(
            cohen_kappa_score(true_levels, predicted_levels, weights="quadratic")
        ),
        "acc_proxy": float(np.mean(difference == 0)),
        "acc_pm1_proxy": float(np.mean(difference <= 1)),
        "level_mae_proxy": float(np.mean(difference)),
        "level_spearman_proxy": level_spearman,
    }


def target_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Return continuous and ordered-proxy metrics for one sensory target."""
    true_array = np.asarray(y_true, dtype=float)
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    metrics = {
        "r2": float(r2_score(true_array, pred_array)),
        "mae": float(mean_absolute_error(true_array, pred_array)),
        "rmse": float(np.sqrt(mean_squared_error(true_array, pred_array))),
        "spearman": _safe_spearman(true_array, pred_array),
    }
    metrics.update(ordered_proxy_metrics(true_array, pred_array))
    return metrics


def matrix_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    """Macro-average all Q1 metrics over the sensory targets."""
    true_array = np.asarray(y_true, dtype=float)
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    if true_array.ndim != 2:
        raise ValueError("prediction matrices must be two-dimensional")
    rows = [
        target_metrics(true_array[:, position], pred_array[:, position])
        for position in range(true_array.shape[1])
    ]
    return {
        name: _nanmean_without_warning([row[name] for row in rows])
        for name in METRIC_COLUMNS
    }


def evaluate_prediction_matrix(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    target_codes: np.ndarray,
    experiment_id: str,
    evaluation: str,
) -> pd.DataFrame:
    """Evaluate every target and return one tidy row per target."""
    true_array = np.asarray(y_true, dtype=float)
```

```
    pred_array = np.asarray(y_pred, dtype=float)
    if true_array.shape != pred_array.shape:
        raise ValueError("y_true and y_pred must have identical shapes")
    if true_array.ndim != 2 or true_array.shape[1] != len(target_codes):
        raise ValueError("target_codes do not match the prediction matrix")
    return pd.DataFrame(
        [
            {
                "experiment_id": experiment_id,
                "evaluation": evaluation,
                "target_code": str(code),
                **target_metrics(true_array[:, position], pred_array[:, position]),
            }
            for position, code in enumerate(target_codes)
        ]
    )


def summarise_target_metrics(by_target: pd.DataFrame) -> pd.DataFrame:
    """Macro-average target metrics for each OOF evaluation."""
    summary = (
        by_target.groupby(["experiment_id", "evaluation"], sort=False)[METRIC_COLUMNS]
        .mean()
        .reset_index()
    )
    summary.insert(2, "n_targets", by_target["target_code"].nunique())
    summary.insert(3, "n_samples", 250)
    return summary
```

requirements.txt

```
numpy==2.2.6
pandas==2.3.3
scipy==1.15.3
scikit-learn==1.7.2
```

run_nnls.py

```
"""完整嵌套交叉验证与新样本推断；默认只显示帮助。"""
from pathlib import Path
import argparse
import json
import numpy as np
import pandas as pd
from stacking_runner import fit_outer_nnls
from q1_metrics import matrix_metrics
from baseline_models import empirical_cdf_grade

def load_tables(folder):
    x=pd.read_csv(folder/'1.csv');y=pd.read_csv(folder/'4.csv')
    for frame in [x,y]:
        if frame.beer.isna().any() or not frame.beer.is_unique:
            raise ValueError('beer 必须完整且唯一')
    if set(x.beer)!=set(y.beer):raise ValueError('样本主键不匹配')
    y=y.set_index('beer').loc[x.beer].reset_index()
    meta=['beer','beer_id','tasting_category_fine']
    if not x.tasting_category_fine.equals(y.tasting_category_fine):raise ValueError('风格不匹配')
    return x,x.drop(columns=meta).to_numpy(float),y.drop(columns=meta)

def main():
    parser=argparse.ArgumentParser(description='感官组收缩 NNLS：复算指标、完整嵌套验证或新样本推断')
```

```
    parser.add_argument('--attachments',type=Path)
    parser.add_argument('--assets',type=Path,default=Path(__file__).resolve().parent.parent/'数据与结果')
    parser.add_argument('--output',type=Path,default=Path('运行结果'))
    parser.add_argument('--execute',action='store_true',help='执行完整 10 次重复 3 折嵌套验证')
    parser.add_argument('--predict-chemistry',type=Path,help='对新化学表预测；仅以现有 250 款训练')
    parser.add_argument('--recompute-oof',type=Path,help='只由已有归档复算，不训练')
    args=parser.parse_args();args.output.mkdir(parents=True,exist_ok=True)
    if args.recompute_oof:
        a=np.load(args.recompute_oof,allow_pickle=False)
        rows=[{'evaluation':f'repeat_{r}',**matrix_metrics(a['y_true'],a['prediction'][r])} for r in
range(10)]
        rows.append({'evaluation':'mean_oof',**matrix_metrics(a['y_true'],a['prediction'].mean(0))})
        pd.DataFrame(rows).to_csv(args.output/'预测指标.csv',index=False)
        return
    if not args.execute and not args.predict_chemistry:parser.print_help();return
    if args.attachments is None:parser.error('需要--attachments 指向官方附件目录')
    chem,x,y_frame=load_tables(args.attachments);y=y_frame.to_numpy(float)
    dictionary=pd.read_csv(args.assets/'target_dictionary.csv').set_index('target_code')
    groups=dictionary.loc[y_frame.columns,'target_group'].astype(str).tolist()
    config=json.loads((Path(__file__).parent/'model_config.json').read_text())
    if args.predict_chemistry:
        new=pd.read_csv(args.predict_chemistry)
        feature_names=chem.drop(columns=['beer','beer_id','tasting_category_fine']).columns
        result=fit_outer_nnls(x,y,new.loc[:,feature_names].to_numpy(float),base_config=config,groups=group
s,scopes=['group'],shrinkage=.5,inner_splits=3,seed=20261023)
        prediction=result['predictions']['group']
        frame=pd.DataFrame(prediction,columns=y_frame.columns);frame.insert(0,'beer',new.beer)
        frame.to_csv(args.output/'连续感官预测.csv',index=False)
        grade=pd.DataFrame(empirical_cdf_grade(y,prediction),columns=y_frame.columns);grade.insert(0,'beer
',new.beer)
        grade.to_csv(args.output/'相对强度预测.csv',index=False)
        return
    splits=pd.read_csv(args.assets/'cross_validation_splits.csv')
    pred=np.full((10,len(x),len(y_frame.columns)),np.nan);folds=np.full((10,len(x)),-1,int);rows=[]
    for repeat in range(10):
        for fold in range(3):
            selected=splits[(splits.repeat==repeat)&(splits.fold==fold)]
            valid=np.sort(selected.row_position.to_numpy(int));train=np.setdiff1d(np.arange(len(x)),valid)
            if not len(valid):raise ValueError('缺少外层划分')
            expected=chem.iloc[selected.row_position].beer.to_numpy()
            np.testing.assert_array_equal(expected,selected.beer.to_numpy())
            result=fit_outer_nnls(x[train],y[train],x[valid],base_config=config,groups=groups,scopes=['gro
up'],shrinkage=.5,inner_splits=3,seed=20261023+repeat*10+fold)
            pred[repeat,valid]=result['predictions']['group'];folds[repeat,valid]=fold
            rows.append({'repeat':repeat,'fold':fold,'weights':result['parameters']['group']})
            print(f'完成重复{repeat+1}、折{fold+1}',flush=True)
    if not np.isfinite(pred).all():raise ValueError('折外预测未完整覆盖')
    np.savez_compressed(args.output/'模型折外预
测.npz',y_true=y,prediction=pred,mean_prediction=pred.mean(0),beer=chem.beer.to_numpy(),style=chem.tasting
_category_fine.to_numpy(str),target_code=y_frame.columns.to_numpy(str),fold_assignment=folds)
    (args.output/'模型权重.json').write_text(json.dumps(rows,ensure_ascii=False,indent=2))

if __name__=='__main__':main()
```

stacking_runner.py

```
from __future__ import annotations
import numpy as np
from sklearn.model_selection import KFold
from baseline_models import fit_predict_extra_trees, fit_predict_ridge_quantile, fit_predict_pls_quantile
from ensemble_models import fit_predict_hgb_multioutput
```

```
from q1_metrics import matrix_metrics
from nnls_stacking import fit_nnls_stacking
def make_inner_splits(n_samples,n_splits,random_state):
    return
list(KFold(n_splits=n_splits,shuffle=True,random_state=random_state).split(np.arange(n_samples)))

def _candidates(config: dict, model: str) -> list[object]:
    if model == "extra_trees":
        return [
            {"max_features": float(max_features), "min_samples_leaf": int(leaf)}
            for max_features in config["extra_trees"]["max_features"]
            for leaf in config["extra_trees"]["min_samples_leaf"]
        ]
    if model == "hgb":
        return list(config["hgb_profiles"])
    if model == "ridge_quantile":
        return [float(value) for value in config["ridge_alphas"]]
    if model == "pls2_quantile":
        return [int(value) for value in config["pls_components"]]
    raise ValueError(f"unsupported base base model: {model}")

def _fit_base(
    model: str,
    parameter: object,
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_validation: np.ndarray,
    config: dict,
    random_state: int,
) -> np.ndarray:
    if model == "extra_trees":
        values = dict(parameter)
        return fit_predict_extra_trees(
            x_train,
            y_train,
            x_validation,
            n_estimators=int(config["extra_trees"]["n_estimators"]),
            max_features=float(values["max_features"]),
            min_samples_leaf=int(values["min_samples_leaf"]),
            max_depth=config["extra_trees"]["max_depth"],
            random_state=random_state,
            n_jobs=1,
        )
    if model == "hgb":
        values = dict(parameter)
        return fit_predict_hgb_multioutput(
            x_train,
            y_train,
            x_validation,
            max_leaf_nodes=int(values["max_leaf_nodes"]),
            min_samples_leaf=int(values["min_samples_leaf"]),
            learning_rate=float(values["learning_rate"]),
            l2_regularization=float(values["l2_regularization"]),
            max_iter=int(values["max_iter"]),
            random_state=random_state,
        )
    if model == "ridge_quantile":
        return fit_predict_ridge_quantile(
            x_train,
            y_train,
            x_validation,
```

```
            alpha=float(parameter),
            random_state=random_state,
        )
    if model == "pls2_quantile":
        return fit_predict_pls_quantile(
            x_train,
            y_train,
            x_validation,
            n_components=int(parameter),
            random_state=random_state,
        )
    raise ValueError(f"unsupported base base model: {model}")

def _parameter_key(metrics: dict[str, float]) -> tuple[float, ...]:
    return (
        metrics["mae"],
        -metrics["r2"],
        -metrics["spearman"],
        -metrics["qwk_proxy"],
        -metrics["acc_pm1_proxy"],
    )

def fit_outer_nnls(x_train, y_train, x_validation, *, base_config, groups,
                   scopes, shrinkage, inner_splits, seed) -> dict:
    """Rebuild base-model OOF entirely inside outer training, then fit NNLS."""
    y = np.asarray(y_train, dtype=float)
    splits = make_inner_splits(len(y), int(inner_splits), int(seed))
    assignment = np.full(len(y), -1, dtype=int)
    for fold, (_, valid) in enumerate(splits):
        assignment[valid] = fold
    inner_predictions, outer_predictions, selections = [], [], {}
    for model_position, model in enumerate(base_config["base_models"]):
        candidate_rows = []
        for candidate_position, parameter in enumerate(_candidates(base_config, model)):
            prediction = np.full_like(y, np.nan)
            for fold, (train, valid) in enumerate(splits):
                prediction[valid] = _fit_base(
                    model, parameter, x_train[train], y[train], x_train[valid], base_config,
                    random_state=int(seed) + model_position * 100000 + candidate_position * 1000 + fold,
                )
            if not np.isfinite(prediction).all():
                raise RuntimeError("stacking base inner OOF is incomplete")
            candidate_rows.append({"parameter": parameter, "prediction": prediction,
                                   "metrics": matrix_metrics(y, prediction)})
        selected = min(candidate_rows, key=lambda row: _parameter_key(row["metrics"]))
        inner_predictions.append(selected["prediction"])
        outer_predictions.append(_fit_base(
            model, selected["parameter"], x_train, y, x_validation, base_config,
            random_state=int(seed) + model_position * 100000,
        ))
        selections[model] = {
            "parameter": selected["parameter"],
            "candidate_metrics": [{"parameter": row["parameter"], "metrics": row["metrics"]}
                                  for row in candidate_rows],
        }
    inner_stack, outer_stack = np.stack(inner_predictions, axis=-1), np.stack(outer_predictions, axis=-1)
    predictions, parameters = {}, {}
    for scope in scopes:
        fitted = fit_nnls_stacking(inner_stack, y, groups=groups, scope=scope, shrinkage=shrinkage)
        predictions[str(scope)] = fitted.predict(outer_stack)
        parameters[str(scope)] = {"weights": fitted.weights.tolist(),
```

```
                                "intercept": fitted.intercept.tolist(),
                                "global_weights": fitted.global_weights.tolist()}
    return {"predictions": predictions, "parameters": parameters, "base_selection": selections,
            "inner_fold_assignment": assignment, "inner_prediction_stack": inner_stack,
            "outer_base_stack": outer_stack}
```